

Datastructuren

Authors

Matt ter Steege
Matttersteege@gmail.com

Contents

1	Invariant	2
2	Zoeken	2
2.1	Binary search	2
3	Sommaties	2
3.1	Rekenkundige reeks	3
3.2	Meetkundige reeks	3
3.3	Harmonische reeks	3
3.4	Rekenregels voor sommaties	4
4	Logaritmen	4
4.1	Belangrijke eigenschappen	4
5	Analyse en de Grote O	4
5.1	Exacte Looptijdanalyse	4
5.2	De Grote O Notatie	5
5.3	Complexiteitsklassen	5
5.4	Variaties op de O	5
6	Recursie	5
6.1	Wat is Recursie?	5
6.2	Verdeel en Heers Aanpak	5
6.3	Voorbeelden	5
6.4	Correctheid Bewijzen	6
6.5	Recurrente Betrekkingen	6
7	Recursieve Algoritmen Analyseren	6
7.1	Recurrente Betrekkingen Oplossen	6
7.2	Asymptotisch Oplossen	6
7.3	Substitutiemethode	6
7.4	Masterstelling	7
8	Kansrekening	7
8.1	Combinatoriek	7

1 | Invariant

Een invariant is een logische uitspraak over variabelen die vóór en na elke iteratie van een loop waar blijft. Het doel is om de correctheid van de loop te structureren. Bij het zoeken naar een geschikte invariant werk je vanuit de gewenste eindtoestand. Vervolgens controleer je:

- Initialisatie: kies beginwaarden (bijv. $i = -1$, $j = n$) zodat de invariant waar is.
- Body: pas i of j zo aan dat de invariant behouden blijft.
- Conditie: stop wanneer $i = j - 1$, zodat invariant $\wedge \neg C$ de gewenste eigenschap geeft.

Daarnaast gebruik je een variant (bijv. $j - i$) die strikt afneemt om terminatie te garanderen.

Voorbeeld (**invariant vinden**):

Gegeven een gesorteerde array A en een waarde x . We willen de eerste index j vinden waarvoor $A[j] \geq x$.

Stap 1: Einddoel formuleren

We willen uiteindelijk:

$$A[j-1] < x \quad \wedge \quad A[j] \geq x$$

Stap 2: Verzwakken naar een invariant

Tijdens de loop is $j - 1$ nog niet vast. Daarom nemen we een algemene vorm:

$$A[i] < x \quad \wedge \quad A[j] \geq x$$

Stap 3: Initialisatie kiezen

Kies $i = -1$ en $j = n$. Met de conventie $A[-1] = -\infty$ en $A[n] = +\infty$ geldt de invariant direct.

Stap 4: Body afleiden

Neem $m = \lfloor (i + j)/2 \rfloor$ ¹.

- Als $A[m] < x$, zet $i = m$
- Anders zet $j = m$

In beide gevallen blijft de invariant behouden.

Stap 5: Stopconditie

Stop als $i = j - 1$. Dan volgt uit de invariant precies de gewenste eigenschap.

2 | Zoeken

Zoeken in een gesorteerde array maakt gebruik van orde: vergelijkingen laten toe om delen van de array uit te sluiten in plaats van lineair te doorlopen.

Binary search

Binary search houdt een interval $[i, j]$ bij waarin de oplossing ligt.

Per iteratie:

- neem $m = \lfloor (i + j)/2 \rfloor$
- als $A[m] < x$, zet $i = m$
- anders zet $j = m$

De invariant garandeert dat de oplossing binnen $[i, j]$ blijft. De variant $j - i$ halveert ongeveer elke stap, wat leidt tot $O(\log n)$ tijd.

3 | Sommaties

Een sommatie heeft de vorm:

$$\sum_{i=p}^q A_i$$

waarbij:

- i : de sommatievariabele (gebonden, mag hernoemd worden)

¹Dit is het binary search algoritme

- p, q : onder- en bovengrens (samen het domein)
- A_i : de termformule, afhankelijk van i

Rekenkundige reeks

Een **rekenkundige rij** heeft een constant verschil tussen opeenvolgende termen. Een rekenkundige reeks is de sommatie van zo'n rij.

$$\sum_{i=p}^q A_i = \frac{\text{Aantal} \times (\text{Eerste} + \text{Laatste})}{2}$$

Afleiding: Stel $S = \sum_{i=0}^n i$. Optellen van S en S omgekeerd:

$$\begin{aligned} S &= 0 + 1 + 2 + \dots + n \\ S &= n + (n-1) + (n-2) + \dots + 0 \\ 2S &= n + n + n + \dots + n = (n+1) \cdot n \end{aligned}$$

Dus: $S = \frac{n(n+1)}{2}$.

Meetkundige reeks

Een **meetkundige rij** heeft een constante verhouding (groeifactor r) tussen opeenvolgende termen. De algemene vorm is:

$$\begin{aligned} \sum_{i=0}^{n-1} A \cdot r^i \\ \sum_{i=0}^{n-1} A \cdot r^i = A \cdot \frac{r^n - 1}{r - 1} = \frac{\text{Volgende} - \text{Eerste}}{\text{Groeifactor} - 1} \end{aligned}$$

Speciaal geval (groeifactor 2):

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

Oneindige meetkundige reeks: Voor $|x| < 1$ geldt:

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$$

Harmonische reeks

Het **harmonisch getal** H_n is gedefinieerd als:

$$H_n = \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

Er bestaat geen gesloten formule voor H_n , maar wel een schatting:

$$\ln(n+1) < H_n < \ln(n) + 1$$

Oftewel: $H_n \approx \ln n$

Voorbeelden:

$$\begin{aligned} \sum_{i=1}^n \frac{1}{2i} &= \frac{1}{2} H_n \quad (\text{som van even breuken}) \\ \sum_{i=1}^n \frac{1}{2i-1} &= H_{2n} - \frac{1}{2} H_n \quad (\text{som van oneven breuken}) \end{aligned}$$

Rekenregels voor sommaties

$$\begin{aligned}\sum_{i=p}^q C \cdot A_i &= C \cdot \sum_{i=p}^q A_i && \text{(constante factor)} \\ \sum_{i=p}^q C &= (q - p + 1) \cdot C && \text{(constante term)} \\ \sum_{i=p}^q (A_i + B_i) &= \sum_{i=p}^q A_i + \sum_{i=p}^q B_i && \text{(termsplitsing)} \\ \sum_{i=p}^q A_i &= \sum_{i=p}^r A_i + \sum_{i=r+1}^q A_i && \text{(domeinsplitsing)} \\ \sum_{i=p}^q A_i &= \sum_{i=p}^{q-1} A_i + A_q && \text{(afsplitsen)} \\ \sum_{i=p}^q A_i &= \sum_{j=p}^q A_j && \text{(hernoemen)} \\ \sum_{i=p}^q A_i &= \sum_{i=\pi(p)}^{\pi(q)} A_{\pi^{-1}(i)} && \text{(dummy transformatie)}\end{aligned}$$

Let op: Bij dummy transformatie moet π een bijectie zijn!

4 | Logaritmen

De **logaritme** $\log_b a$ is de oplossing in x van de vergelijking:

$$b^x = a$$

Belangrijke eigenschappen

$$\begin{aligned}b^{\log_b a} &= a && \text{(definitie)} \\ \log_b 1 &= 0, \quad \log_b b = 1 \\ \log_b (a_1 \cdot a_2) &= \log_b a_1 + \log_b a_2 && \text{(productregel)} \\ \log_b (x^y) &= y \cdot \log_b x && \text{(machtrege)} \\ \log_b a &= \frac{\log_c a}{\log_c b} && \text{(grondtal conversie)} \\ \log_b a &= -\log_b (1/a) \\ \log_b a &= -\log_{1/b} a\end{aligned}$$

5 | Analyse en de Grote O

Exacte Looptijdanalyse

Bij het analyseren van algoritmen tellen we de uitvoeringstijd per instructie. Voor een **for**-lus met n iteraties:

- Initialisatie: 1 keer
- Test: $(n + 1)$ keer
- Increment: n keer
- Body: n keer

Voorbeeld Selection Sort: Voor geneste loops met variabele iteraties gebruiken we sommaties:

$$T(n) = \sum_{i=0}^{n-1} (n-i)(c_1) = \frac{n(n+1)}{2} \cdot c_1 = An^2 + Bn + C$$

De Grote O Notatie

Definitie: $f(n) = O(g(n))$ betekent: $\exists n_0, c > 0 : \forall n > n_0 : f(n) \leq c \cdot g(n)$

Belangrijke eigenschappen:

- $O(2n^2) = O(n^2)$ (constanten mogen weg)
- $n = O(n^2)$ (kleinere termen worden geabsorbeerd)
- $O(n^2) + O(n) = O(n^2)$
- $O(n^2) + O(n^2) = O(n^2)$

Complexiteitsklassen

In oplopende volgorde van groeisnelheid:

Logaritmisch: $\lg n, \lg^2 n, \lg^k n$

- Grondtal maakt niet uit: $O(\lg n) = O(\ln n) = O(\log n)$
- Macht binnen log: $O(\lg(n^2)) = O(\lg n)$
- Macht van log: $O(\lg^2 n) \neq O(\lg n)$

Polynomiaal: $n, n^2, n^3, \sqrt{n}$

- Hoogste exponent is leidend: $n^2 + 2n^3 = O(n^3)$
- Domineert altijd logaritmisch: $\lg^k n = o(n^\varepsilon)$ voor elke $\varepsilon > 0$

Exponentieel: $2^n, 3^n, 5^{n/2}$

- Grootste grondtal is leidend: $2^n + 3^n = O(3^n)$
- Domineert altijd polynomiaal

Variaties op de O

- $O(g(n))$: hoogstens $g(n)$ (bovengrens)
- $\Omega(g(n))$: minstens $g(n)$ (ondergrens): $\exists n_0, c > 0 : \forall n \geq n_0 : f(n) \geq c \cdot g(n)$
- $\Theta(g(n))$: precies $g(n)$ (strakke grens): $f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$

6 | Recursie

Wat is Recursie?

Voor programmeurs: Een functie roept zichzelf aan.

Voor algoritmeontwerp: Je neemt aan dat je kleinere instanties van het probleem al kunt oplossen.

Belangrijke principes:

- Focus op *wat* de methode doet, niet *hoe*
- Schrijf een exacte postconditie op
- Probeer nooit recursie "uit te rollen" in je hoofd

Verdeel en Heers Aanpak

1. Breek het probleem op in kleinere deelproblemen
2. Los elk deelprobleem recursief op
3. Combineer deeloplossingen tot totaaloplossing
4. Definieer een basisgeval voor kleine problemen

Voorbeelden

Torens van Hanoi:

```
Hanoi(n, source, dest):  
    if n == 1:  
        Zet(source, dest)
```

else:

```
Hanoi(n-1, source, temp)
Zet(source, dest)
Hanoi(n-1, temp, dest)
```

Merge Sort:

- Verdeel array in twee helften
- Sorteer beide helften recursief
- Vlecht (merge) beide gesorteerde helften
- Merge-stap kost $\Theta(n)$ tijd

Correctheid Bewijzen

Om een recursief algoritme correct te bewijzen (met inductie):

1. **Variant:** Identificeer maat voor voortgang (bijv. y -waarde), toon aan dat deze bij recursie strikt daalt
2. **Geldigheid:** Toon aan dat recursieve aanroepen voldoen aan preconditionie
3. **Basisgeval:** Bewijs dat base case correct is
4. **Recursief geval:** Neem aan dat recursieve aanroep correct is (IH), bewijs dat huidige aanroep correct is

Recurrente Betrekkingen

Een recurrente betrekking (RB) definieert een functie in termen van kleinere parameters.

Voorbeeld - Hanoi zetten:

$$Z(n) = \begin{cases} 1 & \text{als } n = 1 \\ 2 \cdot Z(n-1) + 1 & \text{als } n > 1 \end{cases}$$

Oplossing: $Z(n) = 2^n - 1$ (bewijs met inductie)

7 | Recursieve Algoritmen Analyseren

Recurrente Betrekkingen Oplossen

Merge Sort RB:

$$M(n) = \begin{cases} O(1) & \text{als } n \leq 1 \\ 2 \cdot M\left(\frac{n}{2}\right) + \Theta(n) & \text{als } n > 1 \end{cases}$$

Asymptotisch Oplossen

Bij asymptotische analyse mogen we negeren:

- De randvoorwaarde (meestal niet belangrijk)
- Afronding bij delen ($\lfloor n/2 \rfloor$ vs $n/2$)
- Exacte constanten in extra termen

Wat WEL belangrijk is: Het aantal recursieve aanroepen!

Substitutiemethode

Raad een oplossing en bewijs deze met inductie.

Voorbeeld: Bewijs dat $M(n) \leq c \cdot n \lg n$ voor $M(n) \leq 2M(n/2) + dn$

Bewijs (inductiestap):

$$\begin{aligned} M(n) &\leq 2M(n/2) + dn \\ &\leq 2 \cdot c \frac{n}{2} \lg \frac{n}{2} + dn \quad (\text{IH}) \\ &= cn(\lg n - 1) + dn \\ &= cn \lg n - n(c - d) \\ &\leq cn \lg n \quad \text{als } c \geq d \end{aligned}$$

Masterstelling

Voor recurrente betrekkingen van de vorm:

$$T(n) = a \cdot T(n/b) + f(n)$$

Stappenplan:

1. Schrijf $f(n) = n^r \cdot \lg^s n$
2. Bereken $\ell = \log_b a$
3. Vergelijk $f(n)$ met n^ℓ :

Geval 1: $r < \ell$ (polynomiaal kleiner) $\Rightarrow T(n) = \Theta(n^\ell)$

Geval 2: $r = \ell$ en $s = 0$ (evenwicht) $\Rightarrow T(n) = \Theta(n^\ell \lg n)$

Geval 3: $r > \ell$ (polynomiaal groter) $\Rightarrow T(n) = \Theta(f(n))$

Voorbeelden:

- Merge sort: $T(n) = 2T(n/2) + \Theta(n)$, dus $a = 2, b = 2, \ell = 1, r = 1 \Rightarrow T(n) = \Theta(n \lg n)$
- Karatsuba: $T(n) = 3T(n/2) + \Theta(n)$, dus $\ell = \log_2 3 \approx 1.58 \Rightarrow T(n) = \Theta(n^{1.58})$

Totale werk: $T(n) = \sum_{i=0}^{\log_b n} a^i \cdot f(n/b^i)$

8 | Kansrekening

Combinatoriek

Faculteit: $n! = 1 \cdot 2 \cdot 3 \cdots n$ met $0! = 1$

Stirling: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$

Permutaties (volgorde belangrijk): $P(n, k) = \frac{n!}{(n-k)!} = n(n-1) \cdots (n-k+1)$

Combinaties (volgorde niet belangrijk): $\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{P(n, k)}{k!}$

Symmetrie: $\binom{n}{k} = \binom{n}{n-k}$